Terminal × pyproject.toml

```
codio@cleverlogo-eventcricket:~/workspace$ poetry install

  RuntimeError

  Poetry could not find a pyproject.toml file in /home/codio/workspace or its parents

  at ~/.local/share/pypoetry/venv/lib/python3.6/site-packages/poetry/core/factory.py:371 in locate
     367|
     368|         else:
     369|             raise RuntimeError(
     370|                 "Poetry could not find a pyproject.toml file in {} or its parents".format(
    → 371|                     cwd
     372|                 )
     373|             )
     374|
codio@cleverlogo-eventcricket:~/workspace$ ls
existing-project  learning-poetry  poetry-installer-error-6flp2ibe.log  poetry-installer-error-bv5kffwr.log  use-requirements
codio@cleverlogo-eventcricket:~/workspace$ cd learning-poetry
codio@cleverlogo-eventcricket:~/workspace/learning-poetry$ poetry add idna
Using version ^3.4 for idna

Updating dependencies
Resolving dependencies... (0.2s)

Writing lock file

Package operations: 1 install, 0 updates, 0 removals

  • Installing idna (3.4)
codio@cleverlogo-eventcricket:~/workspace/learning-poetry$ poetry install
Installing dependencies from lock file

No dependencies to install or update

Installing the current project: learning-poetry (0.1.0)
codio@cleverlogo-eventcricket:~/workspace/learning-poetry$ poetry add Faker --dev
Using version ^19.2.0 for Faker

Updating dependencies
Resolving dependencies... (0.0s)

  SolverProblemError

  The current project's Python requirement (>=3.6,<4.0) is not compatible with some of the required packages Python requirement:
    - faker requires Python >=3.8, so it will not be satisfied for Python >=3.6,<3.8

  Because no versions of faker match >19.2.0,<20.0.0
   and faker (19.2.0) requires Python >=3.8, faker is forbidden.
  So, because learning-poetry depends on Faker (^19.2.0), version solving failed.

  at ~/.local/share/pypoetry/venv/lib/python3.6/site-packages/poetry/puzzle/solver.py:241 in _solve
```

Guide ×

Collapse

The toml File

Metadata

Start by changing into the `learning-poetry` directory.

```
cd learning-poetry
```

When creating a new package, Poetry generates the `pyproject.toml` file. `toml` is a minimalist file format for configuration files. Information is stored as key-value pairs. The goal is to be easy to understand, even if you have never seen a `toml` file before. Click the link below to open the `pyproject.toml` file.

▶ **What does `toml` stand for?**

Open pyproject.toml

The file is divided into four sections denoted by the square brackets. The `[tool.poetry]` section contains the metadata for the project. The name, version, description, and author are all required. Some of this information is filled in automatically for you, while other information needs to be added manually. If you are looking to publish your package to PyPI, you want to fill out as much metadata as you can (see the Poetry documentation). Poetry will automatically take information from `pyproject.toml` and put it into the website for you package on PyPI.

Managing Dependencies

The `pyproject.toml` is central to Poetry's ability to resolve, add, and remove dependencies. The `[tool.poetry.dependencies]` section is the canonical list of dependencies. If your project needs a package, it will be listed here. Click on the link below to open the terminal and enter the command to change to the `learning-poetry` directory and install the `idna` package.

Open terminal

```
poetry add idna
```

Click the link below to open the `pyproject.toml` file once more. You should now see `idna` listed as a dependency.

Open pyproject.toml

▼ **Did you notice?**
When a package is installed from PyPI, a version number is stored. Remember, the `^` means that updates cannot change the left-most, non-zero digit in a version. So Poetry can update `requests` > as long as the version is less than 3. When a package is installed from GitHub, Poetry replaces the version number with the SSH address for the repository.

```
[tool.poetry.dependencies]
python = "^3.6"
shapes = {git = "git@github.com:codio-content/shapes-package.git"}
idna = "^3.3"
```

Let's assume you download a project created in Poetry. You would get all the files and directories in the project. What you would not get are the dependencies. If you run the `install` command, Poetry will look at the `toml` file and install every package listed. This functions very much like a `requirements.txt` file, but you do not need to generate one first; `pyproject.toml` is always kept up-to-date as you add and remove dependencies.

Open terminal

```
poetry install
```

▼ **Did you notice?**
If you run the `install` command Poetry should tell you that there are no dependencies or updates to install. That is because you have already installed all of the dependencies.

The `pyproject.toml` file also has a section for development dependencies. Every Poetry project comes with `pytest` for development purposes. However, the `add` command does not install a package in the development section. The `Faker` package is a fake data generator used by developers. Using the `--dev` flag tells Poetry to install a package in the development section. Let's install `Faker`.

```
poetry add Faker --dev
```

The `pyproject.toml` file should now show `Faker` listed as a development dependency.

Open pyproject.toml

If you have a Poetry project that you would like to use on your own machine, you need the dependencies. However, you only plan on using the package and not developing it. So you do not really need the development dependencies. The command below will install all of the dependencies in the `[tool.poetry.dependencies]` but ignore the dependencies in the `[tool.poetry.dev-dependencies]` section.

```
poetry install --no-dev
```

In a way, Poetry combines the `setup.py` (metadata) file and `requirements.txt` (dependencies) into the `pyproject.toml` file. This is another example of how Poetry has the same capabilities as `pip` but does so in a much more efficient way.

Reading Question

Select all of the true statements about the `pyproject.toml` file.
Hint, there is more than one correct answer.

- [ ] The `pyproject.toml` file stores all of the dependencies for a project.
- [ ] The `pyproject.toml` file stores all of the metadata for a project.
- [ ] The `pyproject.toml` file is another name for the `requirements.txt` file.
- [ ] The `pyproject.toml` file is required for a Poetry project.

The correct answers are:

- The `pyproject.toml` file stores all of the dependencies for a project.
- The `pyproject.toml` file stores all of the metadata for a project.
- The `pyproject.toml` file is required for a Poetry project.

While the `pyproject.toml` file replaces the need for `requirements.txt`, it does more than simply list out all of the dependencies for a project.

Check It!

Next